

WEEK 8

Develop Mini Application using Express and NodeJS

Aim

To develop a mini Online Quiz Application using Node.js and Express.js that allows users to register, log in, take a technical quiz, calculate their score, and review their answers. The application uses HTML, CSS, JavaScript, Express.js and JSON for the frontend, server-side processing and data storage.

Description

The Online Quiz Application is a simple web-based mini application developed using Node.js and Express.js. The application provides a home page, user registration, login, dashboard, quiz, result and answer-review pages. User details are stored in **users.json** and quiz questions are stored in **questions.json**. Express routes handle registration, login, loading questions and submitting the quiz. After submission, the server compares the selected answers with the correct answers and calculates the score. The result is stored temporarily using browser session storage so that the user can review attempted answers.

Application Flow

Step	Page / Operation
1	index.html – Home / Landing Page
2	register.html – User Registration
3	login.html – User Login
4	dashboard.html – Dashboard
5	quiz.html – Technical Quiz
6	result.html – Score Display
7	review.html – Review Attempted Answers
8	Logout – Return to Login Page

Program

1. Project Structure

```
OnlineQuizApplication/  
  ■■■ app.js  
  ■■■ users.json  
  ■■■ questions.json  
  ■■■ package.json  
  ■■■ public/  
    ■■■ index.html  
    ■■■ register.html  
    ■■■ login.html  
    ■■■ dashboard.html
```

```

■■■ quiz.html

■■■ result.html
■■■ review.html
■■■ css/
■   ■■■ style.css
■■■ js/
    ■■■ script.js

```

2. Express / Node.js Server Program – app.js

```

const express = require("express");
const path = require("path"); const
fs = require("fs");

const app = express();
const PORT = 3000;

const usersFile = path.join(__dirname, "users.json"); const
questionsFile = path.join(__dirname, "questions.json");

app.use(express.json()); app.use(express.urlencoded({
extended: true }));
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => {
res.sendFile(path.join(__dirname, "public", "index.html")); });

app.post("/register", (req, res) => {      const { name, age, dob, gender, email,
mobile, username, password } = req.body;

    if (!name || !age || !dob || !gender || !email || !mobile || !username || !password) {
return res.json({ success: false, message: "Please fill all the fields." });      }

    let users = [];      try {          users =
JSON.parse(fs.readFileSync(usersFile, "utf8"));      } catch
(error) {          users = [];      }

    if (users.find(user => user.username === username)) {          return
res.json({ success: false, message: "Username already exists." });      }

    if (users.find(user => user.email === email)) {          return res.json({
success: false, message: "Email already registered." });      }

    users.push({ name, age, dob, gender, email, mobile, username, password });

    fs.writeFileSync(usersFile, JSON.stringify(users, null, 4));

    res.json({ success: true, message: "Registration successful!" });
});

app.post("/login", (req, res) => {      const
{ username, password } = req.body;

    let users = JSON.parse(fs.readFileSync(usersFile, "utf8"));

    const user = users.find(user =>          (user.username ===
username || user.email === username) &&          user.password ===
password      );

    if (!user) {          return res.json({
success: false,          message: "Invalid
username/email or password."      });      }
    res.json({

        success: true,
        message: "Login successful!",
        user: {          name: user.name,
username: user.username,
email: user.email      }
    });
});

```

```

app.get("/api/questions", (req, res) => {
  const questions = JSON.parse(
    fs.readFileSync(questionsFile, "utf8")
  );
  res.json(questions); });

app.post("/submit-quiz", (req, res) => {
  const questions = JSON.parse(
    fs.readFileSync(questionsFile, "utf8")
  );

  const userAnswers = req.body.answers || [];
  let score = 0;

  const results = questions.map((question, index) => {
    const userAnswer = userAnswers[index] || "";
    const isCorrect = userAnswer === question.answer;

    if (isCorrect) score++;

    return {
      question:
        question.question,
      userAnswer: userAnswer,
      correctAnswer:
        question.answer,
      isCorrect
    };
  });

  res.json({
    success:
      true,
    score,
    total: questions.length,
    results
  }); });

app.listen(PORT, () => {
  console.log(`Server running at
  http://localhost:${PORT}`); });

```

3. Client-Side Quiz Logic – script.js

```

// Load questions
const questionsDiv =
  document.getElementById("questions");
let questions = [];

if (questionsDiv) {
  loadQuestions();
}

async function loadQuestions() {
  const response
  = await fetch("/api/questions");
  questions =
  await response.json();

  questionsDiv.innerHTML = "";

  questions.forEach((question, index) => {
    let optionsHTML = "";

    question.options.forEach(option => {
      optionsHTML += `
        <label class="option">

          <input type="radio"
            name="question${index}"
            value="${option}">
            ${option}
          </label>`;
    });

    questionsDiv.innerHTML += `
    <div class="question">
      <h3>${index + 1}. ${question.question}</h3>
      ${optionsHTML}
    </div>`;
  });
}

// Submit quiz
const quizForm =
  document.getElementById("quizForm");

if (quizForm) {
  quizForm.addEventListener("submit", async
  function(event) {
    event.preventDefault();

```

```

        const answers = [];

        questions.forEach((question, index) => {
const selected = document.querySelector(
`input[name="question${index}"]:checked`
        );

        answers.push(selected ? selected.value : "");
    });

    const response = await fetch("/submit-quiz", {
method: "POST",          headers: { "Content-Type":
"application/json" },    body: JSON.stringify({
answers })
    });

    const result = await response.json();

    if (result.success) {
sessionStorage.setItem(
"quizResult",
        JSON.stringify(result)
    );

    window.location.href = "/result.html";
}
    });
}

```

Output

Run the application from the project folder using the following commands:

```

npm install
node app.js

```

The server runs at **http://localhost:3000**. The following output/screenshots can be attached in this section:

S.No.	Output / Screenshot
1	Home / Landing Page
2	Registration Page
3	Login Page
4	Dashboard Page
5	Quiz Questions Page
6	Quiz Result / Score Page
7	Review Answers Page
8	VS Code Terminal showing the running Express server

Welcome Back!

Login to continue your quiz

Username / Email

vaniakkivarapu@gmail.com

Password

.....

Login

Don't have an account? [Register](#)

Welcome, vani 🙌

Ready to test your technical knowledge?

Technical Knowledge Quiz

Test your knowledge of HTML, CSS, JavaScript, Node.js and Express.js.

 **Questions**
10

 **Type**
Multiple Choice

 **Time**
No Limit

 **Start Quiz**



Technical Knowledge Quiz

Select one answer for each question.

1. What does HTML stand for?

- ☐ Hyper Text Markup Language
- ☐ High Text Machine Language
- ☐ Hyperlink Text Markup Language
- ☐ Home Tool Markup Language

2. Which language is used to style web pages?



Quiz Completed!

Here is your quiz result

7 / 10

👍 **Good job! Keep practicing.**

📖 [Review Answers](#)

🔄 [Try Again](#)

🏠 [Dashboard](#)

Result

Thus, the Online Quiz Application was successfully developed as a mini server-side web application using Node.js and Express.js. The application supports user registration and login, loading quiz questions from a JSON file, submitting answers, calculating the score, displaying results, and reviewing attempted answers. The experiment demonstrates the use of Express routing, HTTP POST/GET requests, JSON file handling, Node.js modules, JavaScript, HTML and CSS.